Ejecución de pruebas — EcoResiduos del Llano

1. Cómo se ejecutaron

- **Versión del repositorio:** rama `main`, commit `d5abf77` (PR #27).
- **Comando:** desde la carpeta `backend`, `python -m pytest -v`.
- **Entorno:** Python 3.13, PuLP con el solver CBC, `FUENTE_DATOS=json` (sin conexión a la base de datos).
- **Resultado: 63 pasan, 0 fallan** (7 s).

En las tablas, **Entrada** es el dato con que se llama al código, **Esperado** es lo que exige la prueba y **Obtenido** es lo que devolvió el código en esta ejecución. El *caso base* es la red del proyecto: 10 agroempresas y 4 plantas. El *problema pequeño* es una red de prueba con 3 orígenes y 2 plantas.

2. Modelo exacto (`test_exacto.py`)

Prueba	Entrada	Esperado	Obtenido	¿Correcto?
Costo óptimo	Caso base	Estado Optimal, Z* = 132.532.000 COP/sem, factible y verificado	Optimal, Z* = 132.532.000, factible, verificado	si
Nodo ficticio	Caso base (oferta 1.240 t, capacidad 1.400 t)	Agroempresa ficticia E11 con 160 t, todas hacia P4	E11 con 160 t; holguras [0, 0, 0, 160]	si
Precios sombra	Caso base	u = [44.400 … 79.200]; v = [−29.000, −11.600, −47.200, 0], todos v ≤ 0	Iguales a los esperados	si
Holgura complementaria	Caso base	Costos reducidos ≥ 0; x·costo reducido = 0; v·holgura = 0	Se cumple en todas las rutas y plantas	si
Base y óptimos alternativos	Caso base	14 variables básicas, no degenerada, alternativos E03→P1 y E08→P2	14 básicas, no degenerada, {E03→P1, E08→P2}	Si
Verificación por restricción	Caso base	10 de oferta, 4 de capacidad, 1 de no negatividad, todas cumplen	15 restricciones, todas cumplen	si
Problema pequeño	3 orígenes, 2	Sin nodo ficticio, Z = 260	Sin ficticio, Z =	si

Prueba	Entrada	Esperado	Obtenido	¿Correcto?
balanceado	plantas		260	
Oferta mayor que capacidad	Red con más oferta que capacidad	Planta ficticia P2 con 5 t (lo más caro queda sin procesar)	P2 ficticia, asignación [[10], [5]]	Si
Problema inválido	Datos incoherentes	Se lanza `ProblemaInvalido`	Se lanza `ProblemaInvalido`	si
Id del ficticio	[E01, E02, E10]; [P1…P4]	E11; P5	E11; P5	si
Balancear caso base	Caso base	11 fuentes, la última con 160 t y costos [0, 0, 0, 0]	Igual	si

3. Algoritmo genético (`test_genetico.py`)

Prueba	Entrada	Esperado	Obtenido	¿Correcto?
Resuelve el problema pequeño	Problema pequeño; población 30, 40 generaciones, semilla 1	Solución factible, costo ≥ óptimo exacto (260), cada fila suma su oferta	Factible, costo 265,02, filas [20, 30, 25]	si
Historial de convergencia	Problema pequeño; población 20, 25 generaciones, semilla 3	26 puntos (G + 1), curva que nunca sube, último punto = costo final	26 puntos, de 295,89 a 273,71, nunca sube, último = 273,71	si
Parámetros inválidos	`seleccion="ruleta"`, `tipo_cruce="tres_puntos"`, `poblacion=0`, `tasa_cruce=5`	Pydantic rechaza cada uno con `ValidationError`	Los 4 se rechazan	si

Ejecución adicional (fuera de pytest) con el caso base y los parámetros por defecto (población 100, 500 generaciones, cruce 0,8, mutación 0,05, torneo, semilla 1):

Entrada	Esperado	Obtenido	¿Correcto?
Caso base, semilla 1	Solución factible, costo cercano a Z* (gap pequeño), 501 puntos de historial	Factible, 134.366.553,61 COP/sem, gap 1,38 %, 501 puntos (de 182,9 M a 134,4 M), 3,2 s	si

4. Sensibilidad, red, comparativo, API y base de datos

Prueba	Entrada	Esperado	Obtenido	¿Correcto?
Escenarios de sensibilidad (`test_sensibilidad.py`)	Caso base con los 9 escenarios predefinidos	Base 132.532.000; P4 −30 % no cambia el costo; combustible +10 % mantiene la base; P3 −30 % sube el costo; la red original no cambia	Combustible +10 %: 142.289.200 (+9.757.200, base se mantiene); P3 −30 %: +8.712.000; P4 −30 %: +0; original intacta	si
Combustible solo afecta transporte	Caso base, combustible +10 %	Costo E01→P1 = 25 · 1.320 + 25.000; costo por km original sigue en 1.200	Igual	si
Rangos de sensibilidad	Caso base	Capacidad P1 entre 340 y 530 t; dentro del rango, ΔZ = precio sombra × Δ; ruta básica E02→P1 entre 53.800 y 127.000 (con 126.000 la base se mantiene, con 128.000 no); ruta no básica E01→P1 con costo reducido 39.600 y sin máximo	Igual en los 4 casos	si
Sensibilidad con AG	Combustible +10 % con `incluir_ag=True`	La fila trae el resultado del AG y costo > 0	Trae `genetico` y costo > 0	si
Carga y validación de la red (`test_red.py`, 17 pruebas)	Archivo de configuración del caso base y 9 versiones con errores (valores negativos, ids repetidos, tamaños que no coinciden…)	Matriz de costos igual a la Tabla III del informe; cada error detectado con su mensaje	Matriz igual; los 9 errores detectados	si
Rutas cerradas y penalizaciones	Problema pequeño y caso	Exacto y AG no usan rutas cerradas; si un	Ninguna ruta cerrada usada;	si

Prueba	Entrada	Esperado	Obtenido	¿Correcto?
(`test_rutas_y_penalizaciones.py`, 8 pruebas)	base con rutas deshabilitadas	origen queda sin rutas, error 422 claro; penalizaciones llegan al AG	422 con «A tiene todas sus rutas deshabilitadas»; penalizaciones (7, 9) recibidas	
Comparativo y escalabilidad (`test_comparativo.py`, 6 pruebas)	Gap(110, 100); redes 20×4, 50×6 y 100×10; semilla 7	Gap = 10 %; redes válidas y reproducibles; 80 y 300 variables con costo > 0	Igual	si
API (`test_api.py`, 6 pruebas)	Peticiones a `/salud`, exacto, validar, sensibilidad	`{"estado": "ok"}`; exacto 200 con 132.532.000 y E11; datos inválidos → 422; sensibilidad 200 con 4 rangos de capacidad	Igual	si
CORS (`test_cors.py`, 5 pruebas)	Orígenes localhost:5173, :5174, 127.0.0.1:3000 y un sitio externo	Acepta localhost en cualquier puerto, rechaza el externo	Igual	si

5. Conclusión

Todas las pruebas que se pueden ejecutar sin base de datos pasan (63 de 63), y lo que devuelve el código coincide con lo esperado: el exacto da Z* = 132.532.000 COP/sem con sus precios sombra y su base de 14 rutas, el AG queda factible con un gap de 1,38 % y la sensibilidad respeta los rangos calculados.